

Defensa oral — TPI Food Store (primera entrega)

Guía para defender el proyecto · elevator pitch + preguntas típicas + números medidos

Elevator pitch (30 s): «Food Store es un sistema de gestión de pedidos para un negocio de comidas. Lo modelamos desde el enunciado (5 entidades, 1 relación N:M), lo normalizamos hasta BCNF, lo cargamos con datos masivos (200.000 pedidos, 600.000 líneas) y probamos integridad, concurrencia y optimización contra PostgreSQL 18. La IA generó, pero todo se verificó en el motor: medimos antes/después y descartamos lo que no mejoraba.»

Preguntas típicas de defensa

Modelo de datos (objs 1-4)

- **¿Por qué pedido→producto es N:M?** Un pedido lleva varios productos y un producto se vende en muchos pedidos; se resuelve con detalle_pedido (cantidad, precio_unitario, subtotal — precio congelado, R4).
- **¿Por qué la FK producto→categoría es NOT NULL?** Participación total (R1): todo producto pertenece a una categoría; la categoría tiene participación parcial (puede existir vacía).
- **¿En qué forma normal está el esquema?** BCNF: en la planilla separamos (fecha, cliente, forma_pago) y categoria; el único determinante que queda en cada tabla es su clave candidata.
- **¿Cómo probaste el DDL?** ddl.sql ejecutado de punta a punta sobre una base limpia en PostgreSQL 18, 0 errores (ENUM, IDENTITY, CHECK, UNIQUE, FK con RESTRICT).

Integridad y transacciones (objs 7-8)

- **¿Cómo implementás el borrado lógico?** Columna activo + trigger BEFORE DELETE que lanza excepción; la baja se hace solo con UPDATE activo=FALSE. En la demo: catálogo 50.000→49.999 y el histórico de ventas intacto (12 líneas).
- **¿Qué anomalías de aislamiento reproduciste?** Lectura no repetible (10→50 en READ COMMITTED, estable con REPEATABLE READ) y lectura fantasma (estable con SERIALIZABLE).
- **¿Rompieste las restricciones en algún test?** A propósito: DELETE físico (esperado), stock insuficiente y subtotal incorrecto; todos abortan la transacción y se revierten con ROLLBACK.
- **¿Cuánto tardó el bloqueo FOR UPDATE?** La sesión B quedó esperando ~3,4 s hasta el ROLLBACK de A; también se simuló un deadlock.

Optimización (objs 5-6)

- **¿Qué optimizaste y cuánto ganaste?** C1 facturación: 4387→2052 ms (2,1×); resumen ventas TP3: 3,4×; competencia TP3: 7×; búsqueda de cliente: ~18×; MV de facturación: ~163×.
- **¿Por qué descartaste un índice de la IA?** idx_producto_categoria era redundante con idx_producto_categoria_activo (más costo de escritura, mismo plan); pedido(cliente_id, fecha) no cambió el plan en la competencia.
- **¿Qué es la vista materializada?** Precalcula la facturación por categoría/mes (2447→15 ms). Se refresca con REFRESH CONCURRENTLY mensualmente; entre refrescos queda congelada para reportes.
- **¿Cómo demostraste equivalencia de consultas?** Con EXCEPT = 0 filas entre RANK() y su subconsulta correlacionada, y entre JOIN+GROUP BY y la subconsulta en FROM.

Objetos y procedimientos (objs 5-6)

- **¿Qué objetos programables creaste?** 3 vistas, 3 funciones plpgsql (2 de trigger + fn_total_cliente) y el procedimiento sp_registrar_pedido invocado con CALL.
- **¿Cómo funciona sp_registrar_pedido?** Recibe arrays de productos y cantidades, valida longitudes, inserta el pedido y su detalle en loop y deja que el trigger fn_descontar_stock valide y descuente; todo dentro de la transacción del caller.
- **¿Usas PL/pgSQL en los triggers?** Sí: fn_impedir_delete_logico (RAISE EXCEPTION), fn_descontar_stock (consulta stock y UPDATE), fn_aplicar_subtotal (corrige subtotal automáticamente).

IA (si lo preguntan)

- **¿Qué rol tuvo la IA?** Generó SQL, specs y la interpretación de planes; envió propuestas. Se aceptó solo lo verificado en el motor y se descartó lo redundante o sin impacto.
- **¿Imprecisiones?** La IA cometió 3 al leer un plan (Hash Join ≠ Nested Loop, build/probe, cost estimado ≠ tiempo real) — corregidas contra el plan real.
- **¿Kiro?** No está instalado en la máquina: specs con plantilla de Kiro redactados a mano y OpenCode como IA; constado en cada DUIA.

Requisitos del motor (si lo preguntan)

- **¿Triggers con tablas de transición?** trg_auditar_ajuste_precios usa FOR EACH STATEMENT con REFERENCING OLD/NEW TABLE: un UPDATE de 12.500 precios genera 1 sola fila de auditoría.
- **¿Usás JSONB?** Sí: tabla auxiliar producto_extra con etiquetas/origen/info nutricional; consultas con @>, ->>, -> y cast booleano para filtrar veganos.
- **¿Procedimiento con CALL?** sp_registrar_pedido (PL/pgSQL) recibe arrays, valida e inserta pedido+detalle dejando que los triggers descuenten stock.

Tabla de números para citar

Métrica	Valor
Base de datos	50.000 productos · 20.000 clientes · 200.000 pedidos · 600.000 líneas
C1 facturación (TP4)	4387 → 2052 ms (2,1×)
C2 ranking (TP4)	754 → 751 ms (sin mejora, índice descartado)
Resumen ventas (TP3)	994 → 290 ms (3,4×)
Competencia TP3	39,9 → 5,7 ms (7,0×)
Índice catálogo por nombre	20 → 8,4 ms (Seq→Bitmap, parcial WHERE activo)
Índice cliente por apellido	15,9 → 0,9 ms (~18×)
Pedidos tarjeta por mes	78,4 → 6,8 ms
Vista materializada	2447 → ~15 ms (~163×)
Costo escritura (300 INSERT)	47,5 → 151,6 ms (+3,2×, aceptado)
Concurrencia	B bloqueada ~3,4 s en FOR UPDATE; deadlock simulado
Equivalencias SQL	EXCEPT = 0 filas (RANK vs correlacionada; correlacionada vs GROUP BY)

Guía completa con respuestas por objetivo: 07_defensa_oral.md · Checklist objetivo→evidencia: checklist_tpi.md